VoiceBOT Integration with Vaani-Telephony Services via WebSocket

Table of Contents

1. Introduction

This document provides a detailed integration guide for connecting a third-party VoiceBOT platform to our Vaani-Telephony infrastructure via **WebSocket streaming**. The integration enables real-time audio streaming, conversational AI processing, and synchronized delivery of BOT responses to live telephony sessions.

The goal is to ensure **seamless audio exchange** between the Vaani-Telephony service and the VoiceBOT provider while maintaining reliability, low latency, and secure communication.

---

2. Integration Overview

- **Direction:**
  - Vaani-Telephony → VoiceBOT: Stream inbound audio (caller's speech).
  - VoiceBOT → Vaani-Telephony: Receive audio responses or transcripts from BOT.
- **Protocol:** Secure WebSocket (wss://)
- **Mode:** Real-time bidirectional streaming.
- **Audio Specs:** 8kHz, 16-bit PCM, mono, Base64 encoded.

---

3. Architecture Flow

1. Vaani-Telephony system establishes a secure WebSocket connection to the VoiceBOT endpoint.
2. VoiceBOT responds with a `connected` event.
3. Vaani sends a `start` event with call metadata.
4. Vaani streams caller audio using `media` events.
5. VoiceBOT responds with `media` (BOT audio), `clear` (flush audio), or `transfer` (request agent handoff).
6. Vaani plays BOT audio or executes transfer logic.
7. Either side ends the session with a `stop` event.

---

4. Prerequisites

- WebSocket endpoint provided (e.g., wss://<voicebot-url>/stream/voicebot).
- Network firewall rules allowing secure WebSocket connections (port 443).
- Support for **Base64 audio encoding/decoding**.
- Robust error handling and session management in telephony middleware.

5. WebSocket Events

From Vaani(Telephony) → VoiceBOT

1. **Connected**: Call Initiated from Vaani-telephony platform to VoiceBOT server.

```
{
    "event": "connected",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "connected": {
       "track": "inbound/outbound"
    }
}
```

2. **Start**: Call metadata and media format.

```
{
    "event": "start",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "start": {
        "track": "inbound/outbound"
        "mediaFormat": {
            "encoding": "audio/lin",
            "sampleRate": 8000,
            "channels": 1
        }
    }
}
```

3. **Media**: Audio chunks from caller.

```
{
    "event": "media",
    "media": {
        "chunk": "1", // send chunk number in sequence eg. 1,2,3,4,.....
        "timestamp": "1758021592", // epoch time
        "payload": "no+JhoaJjpz..." // base 64 encoded
    },
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
}
```

4. **Stop**: Call/session termination.

```
{
    "event": "stop",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "stop": {
        "reason": "stop"
    }
```

```
}
```

From VoiceBOT → Vaani(Telephony)

1. **Media**: BOT audio response (Base64 PCM).

```
{
    "event": "media",
    "media": {
        "track": "inbound",
        "chunk": "1", // send chunk number in sequence eg. 1,2,3,4,.....
        "timestamp": "1758021592",  // epoch time
        "payload": "no+JhoaJjpz..." // base 64 encoded
    },
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
}
```

2. **Clear**: Instructs telephony to flush buffered audio.

```
{
    "event": "clear",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
    "clear": {
        "reason": "interrupt"
    }
}
```

3. **Transfer**: Request to hand over the call to a live agent.

```
{
    "event": "transfer",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "transfer": {
        "reason": "transfer",
        "transfer_queue": "queue 1", // optional
        "agent_id": "agent 1" // optional
    }
}
```

4. **Stop**: Call/session termination.

```
{
    "event": "stop",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "stop": {
        "reason": "stop"
    }
}
```

6. Audio Chunk Rules (Bidirectional)

For smooth playback and minimal gaps:

- **Minimum chunk size:** 3.2 KB (~100ms of audio)
- **Maximum chunk size:** 100 KB
- **Chunk size must be a multiple of 320 bytes**

If rules are broken:

- Too small chunks → jitter
- Too large chunks → timeouts
- Not multiple of 320 → playback delays or gaps

---

7. Handling BOT Responses

- Vaani-Telephony must decode BOT's Base64 audio payload into **PCM audio** and stream to the caller.
- If BOT sends `clear`, Vaani-Telephony should immediately stop ongoing playback.
- If BOT sends `transfer`, Vaani-Telephony should execute the agent transfer workflow (see section 8).

---

8. Transfer to Agent

The VoiceBOT can instruct the Vaani-Telephony system to transfer the call to a live agent by sending a `transfer` event.

Example:

```
{
    "event": "transfer",
    "streamSid": "MZXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "transfer": {
        "reason": "transfer",
        "transfer_queue": "queue 1", // optional
        "agent_id": "agent 1" // optional
    }
}
```

- **Reason**: Optional description of why the transfer is requested.
- **Transfer Queue**: Identifier of the destination (e.g., queue, skill group, or direct agent).
- Telephony should interpret this and initiate its standard call transfer procedure.

9. Error Handling & Recovery

- Implement **reconnection logic** if WebSocket drops unexpectedly.
- Retry with exponential backoff.
- Log all failed messages for replay if needed.
- Handle `stop` and `transfer` events gracefully to release or re-route call resources.

---

10. Security Considerations

- Always use **wss://** for encrypted traffic.
- API token must be kept confidential and rotated periodically.
- Restrict WebSocket endpoint access to known IP ranges if possible.

---

11. Testing & Validation

- Validate audio quality (8kHz, 16-bit PCM, mono).
- Test both short utterances and long conversations.
- Simulate network drops and confirm recovery behavior.
- Verify successful call transfer when BOT issues `transfer` event.

---

12. Support

For issues during integration or runtime errors:

- Provide call session IDs (`streamSid`) when raising tickets.
- Maintain logs of all WebSocket exchanges for debugging.